

Invoixy — Technical Architecture & Viva Guide

System Architecture & Examiner Defense Q&A | Invoixy GST SaaS Platform

Document Type: Technical Specification, System Architecture & Viva Defense Guide

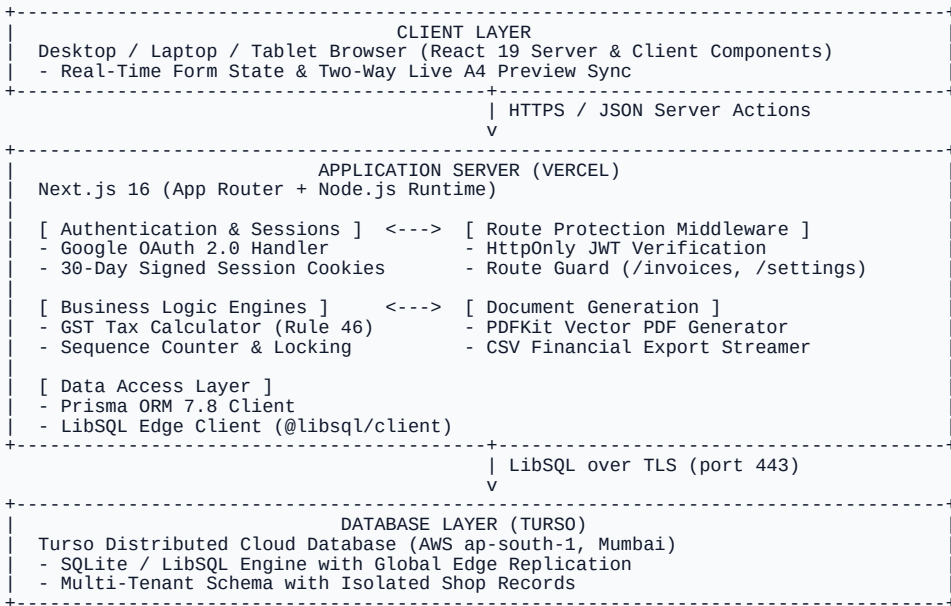
Project: Invoixy (Multi-Tenant GST Invoicing SaaS)

Target Audience: Engineering Evaluators, Project Guides, and Technical Examiners

Deployment: Production on Vercel (`https://invoixy.vercel.app`) with Turso Cloud Database

1. High-Level System Architecture

Invoixy is built on a modern Serverless Cloud Architecture following the Next.js App Router (Full-Stack) paradigm. It combines server-rendered React components with serverless backend API routes and distributed edge database connectivity.

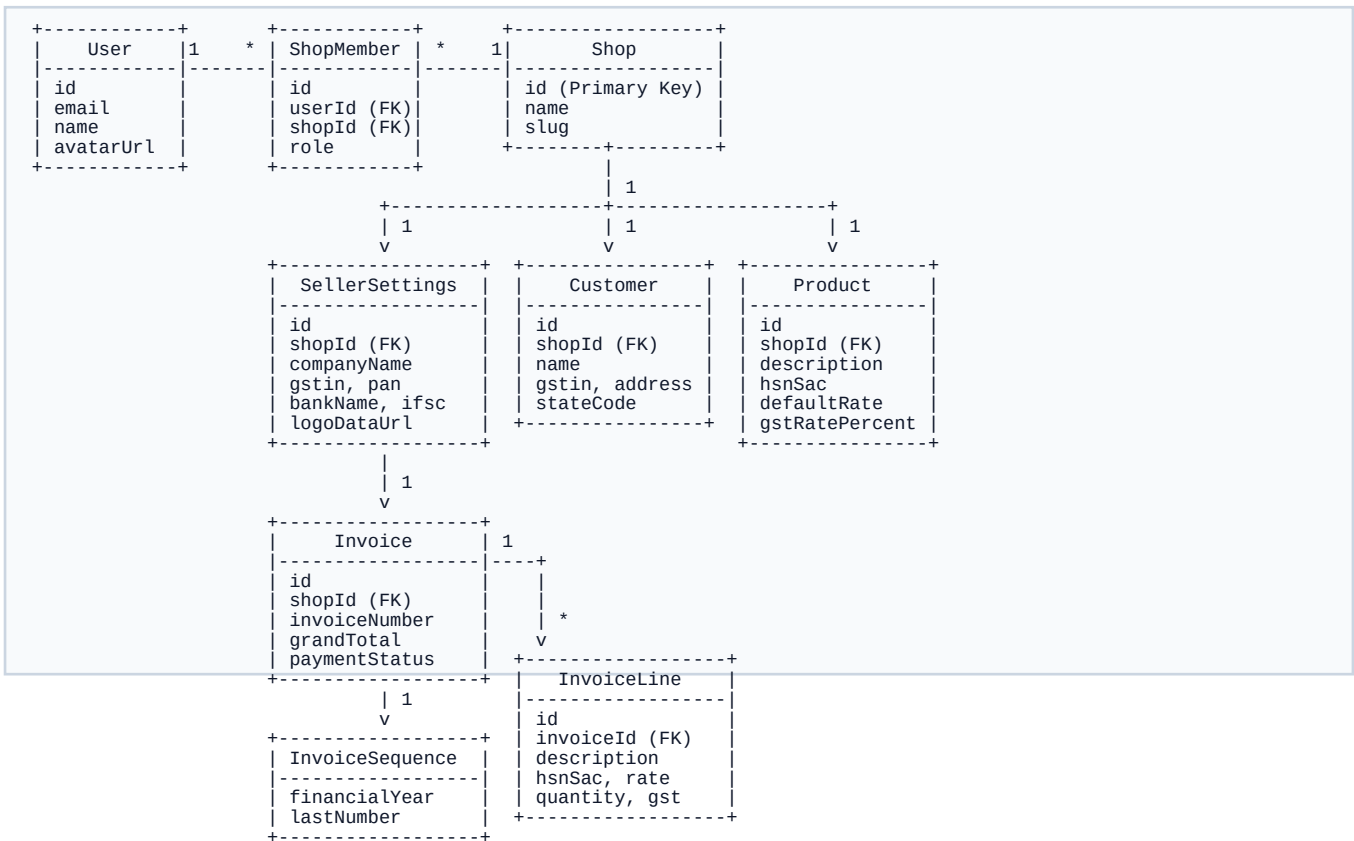


2. Tech Stack Directory & Rationale

Layer	Technology	Version	Why This Tech Was Chosen
Framework	Next.js (App Router)	16.2.6	Combines backend API routes and frontend UI in a single unified codebase with zero boilerplate. Turbopack provides lightning-fast builds.
Language	TypeScript	5.x	Eliminates runtime bugs through static type definitions for invoice drafts, line items, and tax math.
UI Library	React	19.x	Utilizes React Server Components (RSC) for fast initial page load and Client Components for responsive interactive forms.
Styling	Tailwind CSS	4.x	Utility-first CSS ensuring lightweight, zero-runtime styling with consistent spacing and responsive breakpoints.
ORM	Prisma ORM	7.8.0	Type-safe database queries, auto-generated migration schema, and automated TypeScript models.
Cloud Database	Turso (LibSQL)	Cloud	Serverless SQLite engine running on AWS Mumbai (aws-ap-south-1). Provides sub-20ms database queries with zero maintenance.
Local Fallback	SQLite (dev.db)	Local	Allows the entire application to run completely offline on a developer's machine without any internet connection.
PDF Engine	PDFKit	0.17.x	Native Node.js vector drawing engine that generates crisp, high-resolution printable PDFs on the server without needing heavy browser headless instances (like Puppeteer).
Auth & Security	Jose / JWT + Crypto	Built-in	Cryptographically signed, tamper-proof JSON Web Tokens stored in HttpOnly cookies, preventing XSS token theft.
Input Validation	Zod	3.x	Runtime schema validation for seller settings, security credentials, and invoice payload integrity.

3. Database Schema & Data Models (`prisma/schema.prisma`)

The database consists of 10 interconnected relational models:



Detailed Field Breakdown:

- `User`**: Stores authenticated accounts (Google OAuth ID, email, name, avatar).
- `Shop`**: The core tenant container representing a business establishment (*e.g., Apex Electronics & Appliances*).
- `ShopMember`**: Junction table linking Users to Shops with Role-Based Access Control (`OWNER`, `MANAGER`, `MEMBER`).
- `SellerSettings`**: Store identity profile containing business address, PAN, GSTIN, bank details, invoice prefix, and base64 company logo.
- `Customer`**: Pre-saved customer address book with GSTIN and state codes.
- `Product`**: Inventory catalog containing pre-configured descriptions, HSN/SAC codes, base prices, and default GST tax brackets.
- `Invoice`**: Immutable financial snapshot of every issued bill. Contains invoice number, dates, buyer snapshot, tax breakdown, and payment status (`paid`, `part-paid`, `unpaid`).
- `InvoiceLine`**: Line-item table storing each individual item billed on an invoice with its quantity, unit, price, discount %, and calculated tax amounts.
- `InvoiceSequence`**: Thread-safe sequence tracker preventing invoice number collisions across concurrent bill generation.
- `SecuritySettings`**: Stores counter lock hash and inactivity timeout parameters.

4. Multi-Tenant Architecture & Data Isolation

A key technical highlight of Invoxy is its strict multi-tenant isolation:

How Isolation Works:

- Every business resource (`Customer`, `Product`, `Invoice`, `SellerSettings`) has a mandatory foreign key: `shopId String`.
- When a user logs in via Google or Email, the application resolves their `activeShopId` from their signed JWT session cookie.
- Every database query automatically filters by `where: { shopId }`:

```
typescript
// Guaranteed Data Isolation Example
const invoices = await prisma.invoice.findMany({
  where: { shopId: session.activeShopId },
  orderBy: { id: "desc" }
});
```

Dedicated Demo Sandbox Isolation:

- The Problem: In many student demos, when multiple evaluators log into a demo account, they overwrite each other's stores or see personal data.
- Invoxy's Solution: The demo button specifically mounts a static, dedicated demo tenant (`demo\_shop\_03vtyr1s` / *Apex Electronics (Demo)*) seeded with pristine sample invoices, while personal Google user accounts are granted completely separate, private store IDs.

5. Indian GST Calculation Engine (`src/lib/gst.ts`)

Invoxy implements full compliance with Rule 46 of the Central Goods and Services Tax (CGST) Rules:

Mathematical Formulas Implemented:

1. Item Taxable Amount:

$$Taxable\ Amount = Quantity \times Rate \times \left(1 - \frac{Discount\ \%}{100}\right)$$

2. State Code Rule (Intra-State vs. Inter-State):

* If $Seller\ State\ Code == Buyer\ State\ Code$ (e.g., both are 27 - Maharashtra):

$$CGST\ Rate = \frac{GST\ Rate}{2}, \quad SGST\ Rate = \frac{GST\ Rate}{2}, \quad IGST\ Rate = 0$$

$$CGST\ Amount = Taxable\ Amount \times \frac{CGST\ Rate}{100}$$

$$SGST\ Amount = Taxable\ Amount \times \frac{SGST\ Rate}{100}$$

* If $Seller\ State\ Code \neq Buyer\ State\ Code$ (e.g., Seller 27 Maharashtra, Buyer 24 Gujarat):

$$IGST\ Rate = GST\ Rate, \quad CGST\ Rate = 0, \quad SGST\ Rate = 0$$

$$IGST\ Amount = Taxable\ Amount \times \frac{IGST\ Rate}{100}$$

3. Subtotal & Grand Total:

$$Subtotal = \sum Taxable\ Amount$$

$$Total\ Tax = CGST + SGST + IGST$$

$$Unrounded\ Total = Subtotal + Total\ Tax$$

4. Nearest-Rupee Round-Off:

$$Grand\ Total = \text{Math.round}(Unrounded\ Total)$$

$$Round\ Off = Grand\ Total - Unrounded\ Total$$

5. Indian Currency in Words Engine:

* Converts numbers into formal words using the Indian numbering system (*Lakhs* and *Crores* instead of Millions and Billions):

*Example: 12,35,888.00 $\rightarrow$ "INR Two Lakh Thirty-Five Thousand Eight Hundred Eighty-Eight Only".

6. Server-Side PDF Generation Engine (`src/lib/pdf.ts`)

Why Server-Side PDFKit over Browser `window.print()`?

- Browser printing depends on the user's OS, printer margins, zoom level, and screen DPI, often cutting off invoice borders.
- Invoxy uses PDFKit on the Node.js server to generate a standardized A4 document (595.28 x 841.89 points).

- Features of Generated PDF:

1. Base64 company logo decoded directly into an in-memory buffer and rendered into the header.
 2. Tax Invoice header with seller PAN and GSTIN credentials.
 3. Two-box layout for Buyer and Delivery/Consignee addresses.
 4. Itemized line item table with HSN/SAC codes, quantities, rates, and amounts.
 5. GST Tax Breakup sub-table showing exact CGST/SGST/IGST percentages and amounts.
 6. Remittance Box displaying customer bank account and IFSC details.
 7. Amount in words and authorized signatory footer.
-

7. Security, Middleware & Session Management

1. HttpOnly Cookie Architecture:

- * Session tokens are signed using HMAC-SHA256 with AUTH_SECRET.

- * Stored in cookies with HttpOnly: true (prevents JavaScript access, mitigating XSS attacks) and SameSite: Lax (prevents Cross-Site Request Forgery).

2. Reverse Proxy Middleware (`src/proxy.ts`):

- * Intercepts incoming requests.

- * Protects authenticated routes (`/invoices/\*`, `/customers`, `/products`, `/settings`, `/security`).

- * Automatically redirects unauthenticated visitors to `/login`.

3. Auto-Lock Counter Protection (`src/components/AutoLockProvider.tsx`):

- * Listens for user input events (mousemove, keydown, click).

- * Resets an activity timer. If no activity occurs within the configured duration (*e.g., 15 minutes*), the terminal automatically displays a modal lock screen.

8. College Viva / Technical Defense Q&A

Here are the top 10 questions examiners frequently ask, along with the precise technical answers to give:

Q1: What architecture pattern does this application use?

Answer: *"Invoixy follows a Full-Stack Serverless Architecture built on Next.js 16 with React Server Components (RSC) and Server Actions. The frontend handles interactive state and live preview, while Server Actions and API route handlers execute secure business logic and database transactions on the server."*

Q2: Why did you choose Turso and LibSQL instead of traditional MySQL or MongoDB?

Answer: *"We chose Turso because it is a serverless, distributed LibSQL (SQLite fork) database designed for edge environments. It provides sub-20 millisecond query latency in AWS Mumbai, requires zero server maintenance, and supports transactions natively. SQLite's relational model is ideal for structured financial data like invoices and line items."*

Q3: How do you handle multi-tenancy? Can User A see User B's invoices?

Answer: *"No, data isolation is enforced at the database level. Every shop has a globally unique `shopId`. Every query for customers, products, and invoices strictly filters using `where: { shopId }` resolved from the user's cryptographically signed session token. Cross-tenant data leakage is impossible."*

Q4: How does your GST calculation engine handle intra-state vs. inter-state sales?

Answer: *"It compares the seller's state code with the buyer's state code in compliance with Rule 46 of the CGST Act. If both codes match (e.g., 27 for Maharashtra), it automatically divides the GST rate evenly into CGST and SGST. If they differ, it applies the full rate to IGST."*

Q5: How do you prevent invoice number collisions when two invoices are created at the same time?

Answer: *"We use an `InvoiceSequence` table inside an atomic database transaction (`prisma.\$transaction`). The sequence counter for the active financial year is incremented with an atomic upsert lock, ensuring each invoice gets a unique, gapless sequential number."*

Q6: How is the PDF generated, and why not use the browser's native print function?

Answer: *"We generate PDFs server-side using PDFKit. Client-side browser printing depends on local CSS, zoom settings, and printer drivers, which often break page layouts. Generating vector PDFs on the server ensures identical, pixel-perfect, GST-compliant documents on every device."*

Q7: How do you protect authentication sessions?

Answer: *"We use signed JSON Web Tokens (JWT) stored in `HttpOnly`, `SameSite: Lax`, and `Secure` cookies. Because the cookie is `HttpOnly`, client-side malicious scripts cannot access or steal the token."*

Q8: What happens if the internet goes down in a retail store?

Answer: *"Invoixy has a dual-database architecture. In local deployment, it uses a local SQLite file (`dev.db`), allowing shopkeepers to continue generating bills offline. In cloud mode, it connects to Turso."*

Q9: Why did you use Prisma ORM instead of writing raw SQL queries?

Answer: *"Prisma provides compile-time type safety. If a database column changes, TypeScript immediately flags errors throughout the codebase. It also simplifies relational joins between invoices and line items while preventing SQL injection vulnerabilities."*

Q10: What is the purpose of the Demo Store?

Answer: *"The Demo Store provides a zero-friction evaluation sandbox. It mounts a pre-seeded, realistic business environment (*Apex Electronics & Appliances*) so evaluators can test billing, presets, and PDF generation immediately without having to set up a new store from scratch."*

